

■ CYBERSECURITY

Log Analysis & Attack Detection System

Major Project — Technical Documentation

Hybrid Detection
(Rules + ML)

Multi-Layer Log
Ingestion

Attack Timeline
Reconstruction

Real-Time Dashboard

Prepared for Mentor Review • April 2026
B.E. / B.Tech Computer Science — Final Year Major Project

TABLE OF CONTENTS

1	Project Overview & Objective
2	Final System Architecture
3	Technology Stack
4	System Workflow Diagram
5	Phase 0 — Requirements & Setup
6	Phase 1 — Data Pipeline (Ingestion)
7	Phase 2 — Feature Engineering
8	Phase 3 — Detection Engine (Rules + ML)
9	Phase 4 — Correlation Engine
10	Phase 5 — Timeline Mapping
11	Phase 6 — Attack Reconstruction
12	Phase 7 — Dashboard
13	Phase 8 — Multi-Attack Support
14	Phase 9 — Risk Scoring
15	Project Folder Structure
16	Datasets Used
17	What Makes This Professional
18	Build Order Summary

1. PROJECT OVERVIEW & OBJECTIVE

This project builds a **Cybersecurity Log Analysis and Attack Detection System** — a full-stack platform that monitors network and application logs in real time, identifies malicious activity using both rule-based logic and machine learning, and presents actionable intelligence on a live dashboard.

What Problem Does It Solve?

Traditional security tools alert on individual suspicious events but miss the bigger *story* — who attacked, when it started, how it escalated, and what the impact was. This system correlates thousands of individual log events into meaningful **attack sessions** and reconstructs the full timeline of each attack automatically.

Core Capabilities

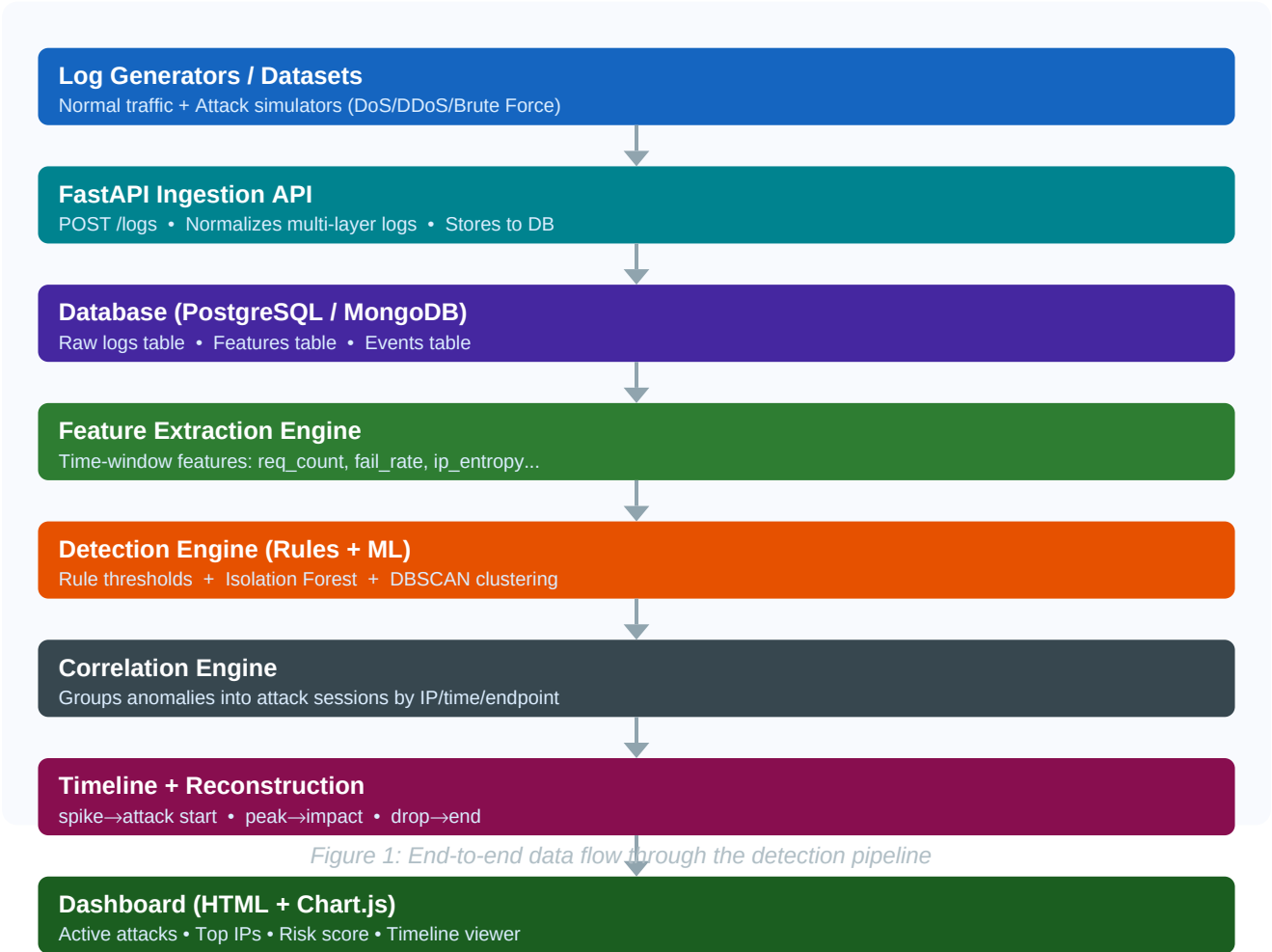
- ✓ **Ingest multi-layer logs**
Accepts IAM, application, network, and system logs through a unified API endpoint (POST /logs).
- ✓ **Extract time-window features**
Groups log events into 5-second windows per IP address and computes statistical features like request count, failure rate, and IP entropy.
- ✓ **Detect anomalies (Hybrid)**
Uses BOTH fast rule-based thresholds AND ML models (Isolation Forest + DBSCAN) for reliable, intelligent detection.
- ✓ **Correlate events into attack groups**
Links individual anomalous events together into unified attack sessions using IP, time window, and endpoint similarity.
- ✓ **Reconstruct attack timelines**
Converts raw correlated data into a human-readable story: spike → attack start, peak → impact, drop → end.
- ✓ **Display attack intelligence**
A real-time web dashboard shows active attacks, top source IPs, risk scores, and full timeline views.

2. FINAL SYSTEM ARCHITECTURE

The system is built as a layered pipeline. Each layer has a single responsibility, making it easy to build, test, and explain separately.

Layer	Component	Responsibility
1	Log Generators / Datasets	Produce normal & attack traffic logs
2	FastAPI Ingestion API	Receive, normalize, and store raw logs
3	Database	Persist logs, features, and events (PostgreSQL / MongoDB)
4	Feature Engine	Build windowed statistical features per IP per 5s
5	Detection Engine	Apply rules + ML to flag anomalies
6	Correlation Engine	Group anomalies into attack sessions
7	Timeline + Reconstruction	Sort events chronologically, build attack story
8	Dashboard	Display live attack intelligence in browser

4. SYSTEM WORKFLOW DIAGRAM



3. TECHNOLOGY STACK

Backend	FastAPI (async)	High-performance Python web framework for building the log ingestion API. Handles concurrent log submissions efficiently.
Database	PostgreSQL / MongoDB	PostgreSQL for structured relational data (logs, features, events). MongoDB as an alternative for flexible JSON-schema logs.
Cache / Stream	Redis (optional)	Optional layer for buffering high-volume log streams in real time before batch processing.
ML Models	scikit-learn	Isolation Forest for unsupervised anomaly scoring. DBSCAN for clustering IPs showing similar attack behaviour.
Data Processing	pandas + numpy	Time-window feature extraction, statistical aggregations, and data cleaning pipelines.
Visualization	HTML + Chart.js	Lightweight dashboard rendered in the browser. Shows charts, tables, timelines, and risk scores without a heavy frontend framework.

5. PHASE 0 — Requirements & Setup

Goal: Set up the development environment and install all dependencies before writing a single line of application code.

Install dependencies:

- pip install fastapi uvicorn psycpg2-binary scikit-learn pandas numpy joblib
- Also install: pymongo (if using MongoDB) and redis (if streaming needed)

Create database schema:

- Table: logs — stores raw normalized log entries
- Table: features — stores computed time-window feature vectors
- Table: events — stores detected anomaly events with scores
- Table: attacks — stores correlated attack sessions

6. PHASE 1 — Data Pipeline (Ingestion)

Goal: Build the entry point of the entire system — an API that continuously receives logs from different sources and stores them in a normalized format.

API Endpoint:

- POST /logs — accepts raw log entries via HTTP

Raw log fields normalized:

- timestamp — when the event occurred
- ip — source IP address of the request
- user — user account involved
- endpoint — which API/URL path was accessed
- status — success or fail
- bytes — payload size
- layer — network / app / system

Log generators (for testing):

- generator.py — produces normal traffic patterns
- dos_attack.py — simulates single-IP flood (DoS)
- ddos_attack.py — simulates distributed flood (DDoS)
- brute_force.py — simulates repeated login failures

7. PHASE 2 — Feature Engineering (Critical)

Goal: Raw logs are not directly useful for ML models. This phase converts raw logs into meaningful numerical features computed over short time windows (e.g., per IP per 5 seconds).

Time-window features computed per IP:

- req_count — total requests in the window (high = flood attack)
- unique_endpoints — how many different URLs were accessed
- fail_rate — ratio of failed to total requests (high = brute force)
- avg_interarrival_time — average gap between requests (low = flood)

- bytes_sum — total bytes transferred in window
- ip_entropy — entropy of source IPs (high = DDoS, many IPs)

Why this matters:

- These 6 features capture what an attack looks like statistically
- They are the input to all ML models in the next phase
- Computed over 5-second windows to catch fast attacks

8. PHASE 3 — Detection Engine (Rules + ML)

Goal: The brain of the system. Uses TWO complementary methods together: fast rule-based checks (reliable, interpretable) and ML models (catches subtle/unknown patterns).

Part A — Rule-Based Detection (Fast & Reliable):

- DoS Rule: req_count > threshold_1 AND same source IP
- DDoS Rule: total_req > threshold_2 AND unique_ips > threshold_3
- Brute Force: fail_rate > 0.7 AND many repeated attempts
- Port Scan: unique_endpoints > threshold in short time

Part B — ML Models:

- Isolation Forest — Input: feature vectors → Output: anomaly score (0 to 1)
- DBSCAN Clustering — Input: [req_count, endpoints, interval] → Output: clusters of IPs behaving similarly (detects distributed attacks)
- Optional: LSTM / ARIMA for time-series trend prediction

Why hybrid is better:

- Rules catch known attack patterns instantly with zero false negatives
- ML catches novel/unknown attacks that rules would miss
- Together they give both speed and intelligence

9. PHASE 4 — Correlation Engine

Goal: Group individual anomalous log events into unified "attack sessions" — connecting the dots to show that many events are part of the same attack.

Grouping keys used:

- IP address (or group of IPs for DDoS)
- Time window — events within the same or adjacent windows
- Endpoint similarity — targeting the same URLs

Logic:

- Group logs where: same time window AND similar behaviour AND anomaly_flag = True
- Output: attack_id, list of IPs involved, all related events

Output example:

- { "attack_id": 1, "ips": ["192.168.1.10"], "events": [...] }

10. PHASE 5 — Timeline Mapping

Goal: Sort all correlated events in chronological order to build a clear sequence of what happened and when during an attack.

Process:

- Sort events by timestamp within each attack session
- Identify key transition points in the event sequence

Timeline example:

- 10:00:00 → Normal traffic (baseline)
- 10:00:03 → Spike begins (first anomaly detected)
- 10:00:05 → Flood peak (maximum attack intensity)
- 10:00:20 → System slowdown (measured impact)

11. PHASE 6 — Attack Reconstruction

Goal: Convert the raw correlated timeline data into a human-readable "attack story" — transforming technical data into actionable intelligence.

Mapping logic:

- spike → attack_start (when the anomaly first appeared)
- peak → impact (the most severe moment of the attack)
- drop → end (when normal traffic resumed)

Output JSON fields:

- start_time, end_time, duration
- timeline — ordered list of key events
- confidence — ML confidence score (e.g., 0.93)
- impact — human label e.g., "High latency" or "Service disruption"

12. PHASE 7 — Dashboard

Goal: A live web interface built with HTML + Chart.js that displays all attack intelligence in one place for security analysts.

Dashboard panels:

- Requests over time — line chart showing traffic volume
- Active attacks — live count of ongoing attack sessions
- Top IPs — table of most suspicious source addresses
- Timeline viewer — scrollable event timeline per attack
- Risk score — per-IP risk rating displayed prominently

13. PHASE 8 — Multi-Attack Support

Goal: Extend the detection engine to handle multiple simultaneous attack types, not just DoS/DDoS.

Attack types supported:

- DoS (Denial of Service) — single IP flood — already implemented
- DDoS (Distributed DoS) — many IPs flood simultaneously
- Brute Force — many failed login attempts from same IP
- Port Scan — rapid sequential access to many different endpoints

14. PHASE 9 — Risk Scoring

Goal: Assign a numerical risk score to each detected attack or suspicious IP so analysts can prioritize their response.

Formula:

- Risk Score = anomaly_score × log(volume) × impact_weight

Factors in the formula:

- anomaly_score — output of the Isolation Forest ML model (0.0 to 1.0)
- log(volume) — log of total request volume (penalizes large floods more)
- impact_weight — user-configured weight for the affected system's importance

Risk Score Formula

$$\text{Risk} = \text{anomaly_score} \times \log(\text{volume}) \times \text{impact_weight}$$

15. PROJECT FOLDER STRUCTURE

```

project/
  app/
    main.py          # FastAPI app entry point
    config.py        # Configuration settings
  api/
    routes_logs.py   # POST /logs – ingestion endpoint
    routes_attack.py  # GET /attacks – attack data API
  db/
    models.py        # ORM schema definitions
    session.py       # Database session manager
  pipeline/
    normalizer.py    # Unify log formats from all sources
    feature.py       # Time-window feature extraction
  detection/
    rules.py         # Threshold-based rule engine
    ml_models.py     # Isolation Forest + DBSCAN
    trainer.py       # ML model training scripts
  correlation/
    grouper.py       # Cluster anomalies into sessions
    timeline.py      # Sort + build event timeline
  reconstruction/
    builder.py       # Convert timeline to attack story
  utils/
    time.py          # Time window utilities
  logs/
    generator.py     # Normal traffic simulator
    dos_attack.py    # DoS attack simulator
    ddos_attack.py   # DDoS attack simulator
    brute_force.py   # Brute force simulator
  data/
    datasets/        # Public training datasets (CICIDS2017 etc.)
    models/          # Saved trained ML models (.pkl files)
  dashboard/
    index.html       # Main dashboard UI
    charts.js        # Chart.js visualizations

```

16. DATASETS USED

Dataset	Type	Used For
CICIDS2017	Public Benchmark	Feature understanding & pre-training experiments
UNSW-NB15	Public Benchmark	Diverse attack patterns for model evaluation
KDD Cup 99	Public (Basic)	Baseline experiments and model benchmarking
Self-Generated Logs	Custom (Main)	Primary training data — labelled normal / dos / ddos / brute_force

17. WHAT MAKES THIS SYSTEM PROFESSIONAL

- Hybrid Detection (Rules + ML)**
Most student projects use only one method. Using both gives better accuracy, speed, and explainability — a key differentiator for industry.
- Multi-Layer Log Support**
Ingests logs from network, application, IAM, and system layers — reflecting real enterprise environments with diverse data sources.
- Time-Window Feature Engineering**
Sophisticated feature engineering over sliding time windows is the same technique used in production SIEM tools like Splunk and QRadar.
- Correlation + Reconstruction**
Turns thousands of raw events into a clear attack narrative — exactly what SOC (Security Operations Center) analysts need.
- Risk Scoring Formula**
Provides a quantitative priority score for each threat, enabling triage — a professional feature missing from most academic projects.
- Self-Generated Labelled Dataset**
Creating your own dataset with realistic attack simulations makes the project reproducible and adds significant academic value.

18. BUILD ORDER SUMMARY

Follow this strict build order — each step depends on the previous one being complete and tested.

Step	Component	What It Does
1	Log Ingestion API	POST /logs endpoint that receives and normalizes raw logs
2	DB Schema	Create tables: logs, features, events, attacks
3	Log Generators	Normal traffic + DoS/DDoS/Brute Force attack simulators
4	Feature Extraction	Time-window feature computation per IP
5	Rule Detection	Threshold-based fast detection engine
6	ML Training + Inference	Train Isolation Forest + DBSCAN, run on feature vectors
7	Correlation Engine	Group anomalies into attack sessions
8	Timeline Mapping	Sort events chronologically within sessions
9	Attack Reconstruction	Build human-readable attack story from timeline
10	Dashboard	Live web UI showing all intelligence in one place

This document covers the complete architecture of the Cybersecurity Log Analysis and Attack Detection System. The system is designed to be built phase by phase, tested at each stage, and demonstrated with real simulated attack traffic on the live dashboard.

— *End of Documentation* —